

Chord DHT 学习系统设计文档

基于 Chord 算法的学习型分布式哈希表系统

文档版本 v1.0

创建日期 2026 年 5 月 27 日

文档状态 正式发布

适用对象 学习分布式哈希表（DHT）的工程师

适用范围 学习用途，不适合直接用于生产环境

协议栈 RESTful HTTP + JSON, SHA-1 (m=160), r=3

目录

第一章：背景与设计目标

- 1.1 Chord 算法简述
- 1.2 设计目标与约束
- 1.3 不在设计范围内

第二章：核心概念与数据结构

- 2.1 ID 空间与一致性哈希
- 2.2 手指表 (Finger Table)
- 2.3 后继列表 (Successor List)
- 2.4 前驱节点 (Predecessor)
- 2.5 节点信息对象 (NodeInfo Schema)

第三章：系统架构

- 3.1 整体架构描述
- 3.2 节点内部模块
- 3.3 节点生命周期状态机
- 3.4 Tracker 的角色与限制

第四章：核心算法详述

- 4.1 加入网络 (Join)
- 4.2 查找后继 (find_successor)
- 4.3 最近前驱 (closest_preceding_node)
- 4.4 稳定化协议 (Stabilize)
- 4.5 通知协议 (Notify)

- 4.6 手指表修复 (fix_fingers)
- 4.7 前驱检查 (check_predecessor)
- 4.8 主动离网 (Graceful Leave)

第五章：周期维护任务

- 5.1 任务调度总览
- 5.2 超时与重试策略
- 5.3 ISOLATED 状态的处理

第六章：边缘情况与处理策略

- 6.1 ~ 6.10 各类边缘场景

第七章：RESTful API 规范

- 7.1 通用约定
- 7.2 统一错误响应格式
- 7.3 节点 API (Node API) — API 1 至 API 10
- 7.4 Tracker API — API 1 至 API 8

第八章：消息时序图（文字描述）

- 8.1 新节点加入网络时序
- 8.2 节点主动离网时序
- 8.3 节点崩溃后的恢复时序

第九章：配置参数参考

第十章：学习扩展建议

- 10.1 下一步实验方向
- 10.2 与真实 DHT 系统的主要差异

第一章

背景与设计目标

1.1 Chord 算法简述

Chord 是由 Stoica 等人于 2001 年在 MIT 提出的一种分布式路由协议，是分布式哈希表（Distributed Hash Table，DHT）领域最具代表性的算法之一。其核心思想是将所有节点和键（Key）均映射到一个统一的环形 ID 空间，通过一致性哈希实现去中心化的分布式路由。

在 Chord 中，ID 空间被定义为 $[0, 2^m)$ 的整数环，本系统中 $m = 160$ ，使用 SHA-1 哈希函数生成节点和键的 ID。每个节点的 ID 由其网络地址（host:port）经 SHA-1 运算得出，表示为 40 位十六进制字符串。

Chord 的关键路由规则基于**顺时针后继原则**：对于给定的键 k ，负责该键的节点是环上 ID 大于等于 k 的第一个节点，称为 k 的后继节点 (Successor)。这意味着每个节点 N 负责区间 $[predecessor.id, N.id]$ 内的所有键。

Chord 的路由效率是其最重要的属性之一：在一个有 N 个节点的网络中，任意键的查找最多需要 $O(\log N)$ 跳，这得益于每个节点维护的手指表 (Finger Table) 结构。手指表中的第 i 个条目指向从当前节点出发、在环上距离 2^i 处的后继节点，使得查找请求每跳都能将剩余距离缩小一半，从而实现对数级的查找效率。

核心性质总结

- **ID 空间**：环形整数空间 $[0, 2^{160})$ ，基于 SHA-1 哈希
- **后继规则**：顺时针方向找到 $ID \geq$ 目标 ID 的第一个节点
- **查找复杂度**： $O(\log N)$ 跳， N 为网络节点总数
- **维护复杂度**：每个节点需维护 $O(\log N)$ 个路由表条目（手指表）
- **容错性**：通过后继列表实现 r 个节点失效的容忍

1.2 设计目标与约束

本系统是一个专为工程师学习 Chord 协议而设计的实现，强调协议语义的正确性和代码的可读性，而非生产级的性能与安全性。以下是本系统明确的设计目标与约束条件：

约束维度	设计决策	选择理由
用途定位	仅组网与路由，不存储 KV 数据	聚焦 Chord 协议本身，降低实现复杂度
通信协议	RESTful HTTP + JSON	无需自定义序列化，调试友好，工具链成熟
维护周期	每 60 秒执行一次 (check_predecessor + stabilize + fix_fingers)	频率适中，便于学习者观察拓扑变化
Tracker 角色	仅负责 bootstrap 引导与全局健康统计，可选组件	Tracker 下线后 DHT 网络完全自治运行
ID 空间	$m = 160$ (SHA-1)	与 Chord 原论文一致，碰撞概率可忽略
后继列表长度	$r = 3$	在容错性和实现复杂度之间取得平衡
手指表长度	$= m = 160$ 个条目	完整实现 Chord 手指表，覆盖全部 ID 空间

1.3 不在设计范围内

为保持系统的学习聚焦性，以下功能明确排除在本系统的设计范围之外。在进行实际开发时，若需要这些功能，应在本系统基础上进行独立的扩展设计：

- **数据存储与 KV 操作**：本系统不实现任何键值存储逻辑，PUT 和 GET 数据操作不在协议中。（参见 10.1 节的扩展建议）

- **TLS / 认证 / 鉴权**：所有 HTTP 通信均为明文，不实现任何身份验证机制。
- **跨数据中心部署**：本系统假设节点在同一局域网或可直接 HTTP 互访的环境中运行。
- **生产级性能优化**：不实现连接池、批量请求、压缩传输等优化手段。
- **Sybil 攻击防御**：节点 ID 由本地 SHA-1 计算，不经过中心化验证。
- **NAT 穿透**：不处理节点位于 NAT 或防火墙后的网络拓扑问题。

核心概念与数据结构

2.1 ID 空间与一致性哈希

Chord 协议的基础是将节点和键统一映射到同一个环形 ID 空间，从而实现去中心化的路由。本系统的 ID 空间定义如下：

ID 生成规则：每个节点的 ID 通过对其规范化 HTTPS URI 字符串执行 SHA-1 哈希运算得到，结果为一个 160 位整数，表示为 40 个十六进制字符的字符串。URI 规范化步骤：① scheme 小写 (https://)；② host 小写；③ 去除尾部斜杠；④ 非默认 443 端口须保留 (如 https://node1.example.com:8443)。例如，对 URI https://node1.example.com 执行 SHA-1 得到的 ID 为 a94a8fe5ccb19ba61c4c0873d391e987982fbbd3。

环形空间：所有 ID 均在模 2^{160} 的整数环上运算。环上的最大值 $2^{160} - 1$ 与 0 在逻辑上相邻，形成一个首尾相连的环形结构。

节点责任区间：节点 N 负责管理所有 ID 落在区间 $(predecessor.id, N.id]$ 内的键。这意味着若一个节点加入或离开，只有其直接相邻区间的数据归属需要重新调整，体现了一致性哈希的核心优势。

以下是本系统使用的两个基础环形运算函数：

函数	定义	用途
<code>clockwise_distance(a, b)</code>	$(b - a) \bmod 2^{160}$	计算从 a 到 b 的顺时针距离（非负整数）
<code>in_range(x, a, b)</code>	x 在环上区间 (a, b] 内	判断 x 是否落在某个环形区间（开闭区间，顺时针方向）

`in_range(x, a, b)` 的实现需处理环绕情况：若 $a < b$ ，则判断 $a < x \leq b$ ；若 $a \geq b$ （区间跨越 0 点），则判断 $x > a$ OR $x \leq b$ 。

2.2 手指表 (Finger Table)

手指表是 Chord 实现 $O(\log N)$ 查找的关键数据结构。每个节点 N 维护一个包含 $m = 160$ 个条目的手指表，使节点能够在路由时快速跳跃到距目标 ID 较近的节点，而不必逐步沿后继链前进。

条目定义：手指表的第 i 个条目 (i 从 0 到 159) 定义为：

- `finger[i].start = (self.id + 2^i) mod 2^160`：这是一个假想的"探针"位置，代表从自身 ID 出发，在环上距离 2^i 处的坐标。
- `finger[i].node = successor(finger[i].start)`：即负责 `finger[i].start` 的实际节点，是该条目存储的路由目标。

条目 i	start 公式	覆盖范围（理论跳跃距离）	路由作用
0	$(N + 2^0) \bmod 2^{160}$	N+1（紧邻后继）	覆盖最小距离，等同于 successor
1	$(N + 2^1) \bmod 2^{160}$	N+2（距离 2）	覆盖约 $1/2^{159}$ 的 ID 空间
79	$(N + 2^{79}) \bmod 2^{160}$	N + 2^{79} （中间位置）	覆盖约一半 ID 空间，最大跳跃中间条目
158	$(N + 2^{158}) \bmod 2^{160}$	N + 2^{158}	覆盖约 1/4 ID 空间
159	$(N + 2^{159}) \bmod 2^{160}$	N + 2^{159} （对角节点）	覆盖约 1/2 ID 空间，最大跳跃条目

路由时使用手指表的方式：在 `find_successor(id)` 中，若目标 id 不在本节点与直接后继之间，则从 `finger[159]` 开始倒序遍历，找到满足 `finger[i].node.id ∈ (self.id, id)` 的最大 i，将请求转发给该节点。这样每一跳都能将剩余距离缩小约一半。

说明

手指表的第 0 个条目（`finger[0].node`）始终等于本节点的直接后继（`successor`），因为 `finger[0].start` 是比自身 ID 大 1 的位置，其后继正是自身的直接后继节点。

2.3 后继列表（Successor List）

后继列表（Successor List）是 Chord 协议中用于增强容错性的关键结构。尽管每个节点在逻辑上只有一个直接后继，但当该后继节点意外崩溃时，若没有备份后继信息，路由链将断裂。后继列表通过维护多个连续后继节点解决这个问题。

本系统参数：后继列表长度 $r = 3$ ，即每个节点维护直接后继之后的 3 个节点的信息。

更新机制：在每次 `stabilize` 周期中，节点 N 从其直接后继 S 获取 S 的后继列表，然后按以下方式构造自己的新后继列表：

1. 取 S 的后继列表的前 $r-1 = 2$ 个节点
2. 将 S 本身放在列表首位
3. 合并得到 N 的新后继列表：`[S, S.successor_list[0], S.successor_list[1]]`

故障切换：当 `stabilize` 发现直接后继 S 无响应时，立即将 `successor_list[1]` 作为新的直接后继，无需等待超时重试，有效避免了环断裂。

去重与验证：更新后继列表时，需去除其中与自身 ID 相同的节点，并对每个候选节点发送 ping 验证其存活状态（可以异步进行，不阻塞主维护流程）。

2.4 前驱节点（Predecessor）

每个节点维护一个指向其直接前驱节点（Predecessor）的引用。前驱节点是环上 ID 比自身小且最近的节点（考虑环绕），即满足 `predecessor.id ∈ (前驱的前驱, self.id)` 的唯一节点。

用途：前驱节点用于 `stabilize` 协议中判断是否有新节点插入，以及 `notify` 协议中更新前驱关系。

predecessor 为 None 的情况：节点在以下情况下 `predecessor` 为 `None`：

- 节点刚刚加入网络，尚未被任何节点通过 `notify` 建立前驱关系
- 节点执行 `check_predecessor` 发现前驱无响应，主动将前驱清空
- 网络中只有一个节点时（单节点环）

处理规则：当 `predecessor` 为 `None` 时，`stabilize` 协议的 `notify` 调用允许任何节点成为自己的前驱（因为无现有前驱可比较）。这是新节点加入后快速建立邻居关系的机制。

2.5 节点信息对象（NodeInfo Schema）

系统中所有 API 交互使用统一的节点描述对象（NodeInfo），定义如下：

```
{
  "node_id":    "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3", // 40 位十六进制字符串，SHA-1(uri) 的结果
  "uri":        "https://node1.example.com",                // 节点的 HTTPS URI（规范化，小写，无尾部斜杠）
  "status":     "ACTIVE",                                    // 节点状态（见 3.3 节）
  "joined_at":  "2026-05-27T12:00:00Z"                      // 加入网络的时间（ISO 8601 UTC）
}
```

字段	类型	必填	说明
node_id	string	是	40 位十六进制字符串，SHA-1(uri) 的结果
uri	string	是	节点的完整 HTTPS URI，格式为 <code>https://hostname</code> （或 <code>https://hostname:port</code> 若非 443 端口）。用于计算 <code>node_id</code> 并对外通告，必须规范化：scheme 和 host 均小写，去除尾部斜杠
status	string	是	INITIALIZING / JOINING / ACTIVE / LEAVING / ISOLATED
joined_at	string	否	节点成功完成 join 流程的时间戳（ISO 8601 UTC）
last_seen	string	否	Tracker 记录该节点最后一次心跳的时间（仅 Tracker 使用）

在节点间的路由 API 中（如 `find_successor` 响应、`notify` 请求等），`NodeInfo` 对象通常只包含 `node_id` 和 `uri` 两个核心字段，`status` 和 `joined_at` 为可选附加字段。

系统架构

3.1 整体架构描述

本系统由两类组件构成：可选的中心化 **Tracker 服务**，以及多个完全去中心化的 **Node 服务**。两者通过 RESTful HTTP 通信，Tracker 仅在节点引导（bootstrap）阶段发挥作用，DHT 路由网络的正常运行完全不依赖 Tracker。

Tracker 服务：单一的中心化服务，负责维护一份已知活跃节点的名册，供新节点加入时获取初始连接点（seed 节点）。Tracker 不参与任何路由决策，不存储路由信息，也不对节点状态进行主动探测——它仅被动接收节点的注册和心跳上报，并提供全局健康统计接口供监控面板调用。

Node 服务（DHT 节点）：每个节点都是对等的（P2P），运行相同的协议实现。节点之间通过 RESTful HTTP 直接通信，不经过 Tracker 中转。每个节点独立维护自己的手指表、后继列表和前驱信息，并通过周期性维护任务保持路由表的准确性。

网络自治性的保证：Tracker 下线后，现有节点完全不受影响，原因如下：

- 每个节点维护 $r=3$ 个后继节点的备份，即使直接后继崩溃也能从列表中恢复
- stabilize 协议每分钟运行一次，持续修复因节点加入/离开导致的路由不一致
- check_predecessor 每分钟检测前驱存活，及时清空失效前驱以接受新的前驱关系
- fix_fingers 轮转修复手指表，确保路由效率在 $O(\log N)$ 级别

新节点加入流程概述：新节点启动后，首先向 Tracker 请求 seed 节点列表，然后选择一个 seed 节点作为“引路人”，发起 find_successor 请求定位自己在环上的位置，完成初始化后通过 notify 协议向邻居节点通报自己的存在。

3.2 节点内部模块

每个 Node 服务在内部由以下四个模块组成，模块间通过内部函数调用交互，对外只暴露 HTTP API：

模块编号	模块名称	职责描述
1	HTTP Server	监听本节点的 HTTP 端口，接受来自其他节点的路由请求（find_successor、notify、join 等）和维护请求（ping、predecessor 查询等）。负责解析请求体、调用 Chord 状态机的对应方法，并序列化响应。
2	Chord 状态机	维护节点的核心 Chord 状态：node_id、predecessor、finger_table[0..159]、successor_list[0..2]、status。实现所有 Chord 协议算法（find_successor、closest_preceding_node、notify 逻辑等）。状态的读写需注意并发安全（使用读写锁）。
3	周期任务调度器	每 60 秒触发一轮维护任务，按序执行：check_predecessor → stabilize → fix_fingers → health_check_ring → report_to_tracker。调度器应确保同一时刻只有一个维护轮次在运行，避免并发维护导致状态混乱。

4	Tracker 客户端（可选）	负责向 Tracker 注册自身、发送心跳、上报状态统计，以及在 join 阶段获取 seed 节点列表。该模块完全可选：若未配置 Tracker 地址，则跳过所有与 Tracker 交互的步骤，依靠手动配置的 seed 节点完成 bootstrap。
---	-----------------	--

3.3 节点生命周期状态机

每个节点在其运行生命周期内经历以下五种状态。状态的变迁由节点自身的运行逻辑驱动，不存在外部的状态控制器：

状态名称	含义	此时节点行为
INITIALIZING	节点启动，正在执行初始化	计算自身 node_id，初始化数据结构（finger_table 全指向自身，predecessor=None，successor=self），尚未对外提供服务
JOINING	正在执行 join 流程	向 seed 节点发起 find_successor，设置初始 successor 和 successor_list，向 successor 发送 notify。此时不响应其他节点的路由请求
ACTIVE	正常运行状态	完整响应所有 API 请求，参与路由转发，执行周期维护任务，向 Tracker 发送心跳
LEAVING	主动离网，正在执行 graceful leave	拒绝新的路由请求（返回 503 NODE_LEAVING），正在通知邻居节点并向 Tracker 注销
ISOLATED	孤立状态，无法联系任何邻居	停止路由服务（find_successor 返回 503），继续响应 ping 和 identity 请求，持续尝试通过 Tracker 或其他途径重新加入网络

状态转移规则如下表所示：

当前状态	触发事件	下一状态	说明
INITIALIZING	初始化完成，开始 join 流程	JOINING	无论有无 seed 节点均进入此状态
JOINING	join 流程成功完成（notify 已发送）	ACTIVE	正常启动路径
JOINING	所有 seed 节点均不可达	ACTIVE（单节点环）	N 自成环，等待其他节点加入
ACTIVE	收到用户/系统的关闭信号	LEAVING	触发 graceful leave 流程
ACTIVE	successor_list 中所有节点均无响应	ISOLATED	维护周期中检测到环断裂
LEAVING	graceful leave 流程完成	—（进程退出）	节点停止 HTTP 服务并退出
ISOLATED	成功联系到任意其他节点	JOINING	执行完整 join 流程重新加入
ISOLATED	收到用户/系统的关闭信号	—（进程退出）	跳过 graceful leave，直接退出

3.4 Tracker 的角色与限制

Tracker 在本系统中是一个轻量级的可选辅助组件，其职责和限制需清晰理解，以避免对系统行为产生误解：

Tracker 存储的数据：Tracker 维护一份已知节点的记录表，每条记录包含节点的 NodeInfo 以及最后汇报时间 (last_seen)。Tracker 存储的最大节点数可配置（默认 1000），超过上限时淘汰最久未活跃的节点。

Tracker 不做的事情：Tracker 不存储任何路由信息（手指表、后继列表），不参与任何 find_successor 查找，不主动探测节点存活状态（仅依赖节点的 heartbeat 上报）。

Tracker 提供的服务：

- Bootstrap 服务：随机返回 k 个已知活跃节点作为新节点的 seed（倾向于选择 last_seen 时间最近的节点）
- 全局健康统计：汇总所有节点的状态分布、手指表覆盖率、节点存活时长等指标，供监控面板调用
- 节点注册与注销：记录节点的加入和主动离网事件

Tracker 下线后的影响：

- 对现有 ACTIVE 节点：**无影响**。节点的 report_to_tracker 任务会静默失败，不影响 stabilize 和 fix_fingers 的执行。
- 对新加入节点：**无法通过 Tracker 获取 seed 节点**，必须通过手动配置的 seed 节点 URI（直接指定任意已知节点的 https://hostname）进行 bootstrap。
- 对监控面板：无法获取全局健康统计，但可以直接查询各节点的 /chord/state 接口获取本地状态。

第四章

核心算法详述

4.1 加入网络 (Join)

节点 N 加入 Chord 网络的完整流程分为以下步骤：

步骤 1：节点 N 启动，计算 `node_id = SHA1(uri)`（对节点 URI 规范化后执行 SHA-1），将结果转换为 40 位十六进制字符串。初始化内部状态：`finger_table` 全部指向自身，`predecessor = None`，`successor = self`，`successor_list = [self]`，`status = INITIALIZING`。

步骤 2：若配置了 Tracker 地址，N 向 Tracker 请求 k=5 个 seed 节点（GET /tracker/nodes/seeds?count=5&exclude={N.id}）。若未配置 Tracker，则使用手动配置的 seed 节点列表。

步骤 3（无 seed 节点——N 是网络中第一个节点）：N 自成单节点环，`finger_table` 全部指向自身，`predecessor = None`，`successor = self`，`successor_list = [self]`，状态直接切换为 ACTIVE。向 Tracker 注册（如有）。等待其他节点加入。

步骤 4（有 seed 节点 S）：N 状态切换为 JOINING，依次执行：

- **步骤 4a：**N 向 S 发起 POST /chord/join（传入 N 的 NodeInfo），S 内部调用 `find_successor(N.id)`，返回 N 的初始 successor（记为 S'）及其 `successor_list`。
- **步骤 4b：**N 设置 `successor = S'`，`predecessor = None`。

- **步骤 4c:** N 使用 S' 返回的 successor_list 初始化自己的 successor_list (取前 r=3 个)。
- **步骤 4d:** N 向 S' 发送 POST /chord/notify (通知 S': N 可能是你的前驱)。
- **步骤 4e:** N 状态切换为 ACTIVE, 开始响应外部请求, 启动周期维护调度器。
- **步骤 4f:** N 向 Tracker 注册 (POST /tracker/nodes, 如有 Tracker)。

步骤 5 (边缘情况: 超时处理): 若步骤 4a 中 S 超时或返回错误, N 依次尝试 seed 列表中的下一个节点 S2、S3..... 直至成功或列表耗尽。若所有 seed 均失败, N 退化为单节点环 (步骤 3 的逻辑)。

4.2 查找后继 (find_successor)

find_successor 是 Chord 路由的核心操作。本系统采用**迭代模式** (Iterative Lookup), 而非递归模式, 原因是迭代模式避免了 HTTP 调用链的深度嵌套, 调用方自行维护跳转状态, 使路由过程更易于调试和追踪。

迭代模式的工作原理: 发起方 (通常是 join 中的新节点或需要路由的节点) 向第一个候选节点发起 find_successor 请求。若该节点知道答案, 直接返回 found: true 和结果; 若不知道, 返回 found: false 和下一跳 (next_hop) 节点信息。发起方收到 next_hop 后, 向 next_hop 节点再次发起请求, 如此循环, 直到 found: true 或超过 max_hops。

单节点执行逻辑 (收到 find_successor(id) 请求时):

1. 检查 hop_count >= max_hops (161): 若超过, 返回 503 MAX_HOPS_EXCEEDED。
2. 检查 visited_nodes 是否包含自身 ID: 若是, 说明出现循环, 返回错误。
3. 检查 id ∈ (self.id, self.successor.id]: 若是, 直接返回 {found: true, successor: self.successor}。
4. 否则, 调用 closest_preceding_node(id) 找到最接近 id 的候选节点 F。
5. 若 F == self (无更好的候选): 返回 {found: true, successor: self.successor} (退化返回)。
6. 否则: 返回 {found: false, next_hop: F, hop_count: hop_count + 1}。

重要: 迭代模式的关键区别

在迭代模式下, 节点收到 find_successor 请求后, **绝对不会自己去调用 next_hop 节点**, 而是直接将 next_hop 返回给调用方。这与递归模式的根本区别在于: 调用链的控制权始终在发起方, HTTP 调用深度永远为 1, 而非 $O(\log N)$ 层嵌套。

4.3 最近前驱 (closest_preceding_node)

closest_preceding_node(id) 用于在本地路由表中找到最适合转发 find_successor(id) 请求的候选节点, 即环上位于 (self.id, id) 之间、距 id 最近的已知节点。

搜索顺序:

1. 从手指表最高位 finger[159] 开始, 倒序遍历到 finger[0]。
2. 对每个条目, 检查 finger[i].node.id ∈ (self.id, id), 且该节点状态不为 failed。

3. 找到第一个满足条件的条目，返回 `finger[i].node`。
4. 若手指表中无合适候选，遍历后继列表 (`successor_list`)，同样倒序检查。
5. 若仍无候选，返回 `self` (发起方将退化为线性前进)。

后继列表的补充作用：后继列表中的节点在环上比手指表更靠近自身，能补充手指表在短距离上的精度盲区（尤其在手指表条目尚未全部修复时）。先查手指表再查后继列表，能在两种数据源之间取得最优的跳跃选择。

4.4 稳定化协议 (Stabilize)

Stabilize 是 Chord 网络自愈能力的核心机制，每 60 秒由调度器触发一次，执行以下步骤：

1. 向当前 `successor` 发送 `GET /chord/predecessor`，获取 `successor` 的前驱节点（记为 `x`）。
2. 若 `x != None` 且 `x.id ∈ (self.id, successor.id)`：说明有新节点 `x` 已插入自己与 `successor` 之间，更新 `successor = x`。这是 stabilize 发现并接纳新邻居的关键步骤。
3. 向（可能已更新的）`successor` 发送 `POST /chord/notify(self)`，让 `successor` 更新其 `predecessor` 关系。
4. 向 `successor` 发送 `GET /chord/successor_list`，获取其后继列表，更新自己的 `successor_list`（取 `successor` 的前 `r-1` 个后继，加上 `successor` 自身）。
5. 若步骤 1 的请求中 `successor` 无响应（超时或错误）：从 `successor_list` 中取下一个节点作为新的 `successor`，重复步骤 1-4。若 `successor_list` 也全部无响应，则节点切换为 `ISOLATED` 状态。

4.5 通知协议 (Notify)

Notify 是 stabilize 协议的配套操作：节点 `N` 向其 `successor` 发送 `notify`，告知“我可能是你的前驱”。接收方（设为节点 `M`）执行以下逻辑：

- 若 `M.predecessor == None`：无条件接受，设置 `predecessor = N`，返回 `{accepted: true}`。
- 若 `N.id ∈ (M.predecessor.id, M.id)`：N 比现有前驱更近，更新 `predecessor = N`，返回 `{accepted: true}`。
- 否则：N 不在合法前驱范围内（可能是过期消息或并发 `notify`），忽略，返回 `{accepted: false}`，`predecessor` 不变。

Notify 是幂等的：对同一节点多次 `notify` 不会产生副作用。这是 Chord 协议容忍并发操作的重要基础。

4.6 手指表修复 (fix_fingers)

`fix_fingers` 确保手指表中的条目保持准确，使路由效率维持在 $O(\log N)$ 级别。与 stabilize 不同，`fix_fingers` 采用**轮转修复策略**：每次维护周期只修复一个条目，而非全部 160 个。

执行步骤：

1. 读取当前的 `next_finger_index`（初始为 0，每次执行后递增，到 159 后回绕为 0）。
2. 计算 `finger[next_finger_index].start = (self.id + 2next_finger_index) mod 2160`。
3. 对该 `start` 值执行 `find_successor(start)`（迭代模式，通过现有路由表发起请求）。

4. 将查找结果更新到 `finger[next_finger_index].node`，状态标记为 `ok`。
5. 递增 `next_finger_index`，供下一次维护周期使用。

为什么不一次修复全部 160 个条目？

若每分钟修复全部 160 个手指表条目，一个有 N 个节点的网络每分钟将产生 $N \times 160$ 次 `find_successor` 请求，在节点较多时会形成显著的网络放大效应。轮转策略每分钟每节点只发起 1 次 `find_successor`，手指表在 160 分钟（约 2.7 小时）内完整刷新一轮，在学习场景下完全可接受。

4.7 前驱检查（`check_predecessor`）

`check_predecessor` 在每个维护周期的最开始执行（先于 `stabilize`），目的是及时清空失效的前驱节点，为 `stabilize` 中的 `notify` 逻辑准备好正确的状态：

1. 若 `predecessor == None`：直接跳过（无需检查）。
2. 向 `predecessor` 发送 `GET /chord/ping`。
3. 若 5 秒内收到正常响应：前驱存活，无操作。
4. 若超时或返回错误：将 `predecessor` 设置为 `None`。下一次 `stabilize` 执行时，若有新的合法节点通过 `notify` 请求加入，可以无阻碍地成为新前驱。

4.8 主动离网（Graceful Leave）

当节点 N 收到关闭信号时，应尽可能地通知邻居节点以减少拓扑修复的延迟，而非直接崩溃退出：

1. N 状态切换为 `LEAVING`，停止接受新的路由请求（返回 `503 NODE_LEAVING`）。
2. N 向 `successor (A)` 发送 `POST /chord/leave (role: "predecessor_leaving", new_predecessor: N.predecessor)`，通知 A 其前驱已变更为 N 的前驱节点 P 。 A 收到后立即更新 `predecessor` 并触发一次提前的 `stabilize`。
3. N 向 `predecessor (P)` 发送 `POST /chord/leave (role: "successor_leaving", new_successor: A)`，通知 P 其后继已变更为 A 。 P 收到后立即更新 `successor` 并触发一次提前的 `stabilize`。
4. N 向 `Tracker` 发送 `DELETE /tracker/nodes/{N.id}` 注销（若有 `Tracker`）。
5. N 停止 `HTTP` 服务，进程退出。

容错说明

即使步骤 2 或步骤 3 因网络问题失败，Chord 协议的自愈机制也能在下一个维护周期（60 秒内）自动修复拓扑：`successor` 的 `check_predecessor` 会发现前驱失效并清空，`stabilize` 会发现新的合法前驱。Graceful Leave 的意义在于将修复时间从约 2 分钟缩短到几秒。

周期维护任务

5.1 任务调度总览

每个 ACTIVE 节点每 60 秒执行一轮完整的维护任务，任务按固定顺序串行执行，确保状态更新的一致性。以下是每轮维护的完整任务清单：

顺序	任务名称	说明	预期耗时	依赖关系
1	check_predecessor	Ping 前驱节点，若无响应则将 predecessor 置为 None，为后续 notify 清理障碍	< 5 秒	无（首位执行）
2	stabilize	询问 successor 的前驱、更新 successor、发送 notify、刷新 successor_list	< 10 秒	需在 check_predecessor 之后执行
3	fix_fingers	轮转修复当前 next_finger_index 位置的手指表条目（每轮仅修复一个）	< 5 秒	需要有效的 successor 信息
4	health_check_ring	对 successor_list 中的所有节点（最多 3 个）发送 ping，更新其存活状态标记	< 15 秒	可并行化执行（3 个 ping 同时发出）
5	report_to_tracker	向 Tracker 发送 heartbeat，上报本节点状态和统计信息（Tracker 不可达时静默跳过）	< 5 秒	完全可选，失败不影响其他任务

每轮维护的最大预期总耗时：约 30-40 秒（任务 4 的三个 ping 并行时）。调度器在上一轮完成后等待剩余时间（60 - 实际耗时），确保维护周期大致为 60 秒。

5.2 超时与重试策略

网络请求的超时和失败处理策略直接影响系统对节点失效的检测速度和准确性。本系统采用以下策略：

场景	策略	说明
单次 HTTP 请求超时	5 秒硬超时	超过 5 秒未响应，视为本次请求失败
第 1 次请求失败	标记为 suspicious	节点可能临时高负载或网络抖动，降低路由优先级但不移除
连续 3 次请求失败	标记为 failed	从手指表和后继列表中移除，不再向其发送路由请求
suspicious 节点	路由中降低优先级	closest_preceding_node 中优先选择 ok 状态的节点，suspicious 节点作为备选
failed 节点	从路由表移除	从手指表条目和后继列表中删除，下次 fix_fingers 会用新节点填充
维护任务中的失败	不重试，等待下一周期	维护任务的设计假设是"最终一致"，单次失败不触发即时重试

状态恢复：一个曾被标记为 `suspicious` 或 `failed` 的节点，若在后续的 `fix_fingers` 或 `health_check_ring` 中被成功 ping 通，其状态将重置为 `ok`。这处理了节点临时网络问题后恢复的场景。

5.3 ISOLATED 状态的处理

当一个节点在 `stabilize` 过程中发现其 `successor_list` 中的所有节点均无响应时，该节点切换为 `ISOLATED` 状态，并按以下逻辑处理：

1. 节点切换为 `ISOLATED` 状态，停止对外的路由服务（`find_successor` 接口返回 `503 NODE_ISOLATED`）。
2. 继续响应 `/chord/ping` 和 `/chord/identity` 请求（用于诊断和被其他节点发现）。
3. 每个维护周期尝试联系 `Tracker`，获取新的 `seed` 节点列表。
4. 若 `Tracker` 也不可达：节点自我降级为单节点环，`predecessor = None`，`successor = self`，等待手动干预或其他节点主动联系。
5. 一旦收到任意来自其他节点的 `notify` 请求，或成功通过 `Tracker` 获取到可达的 `seed` 节点：切换回 `JOINING` 状态，执行完整的 `join` 流程重新加入网络。

注意

`ISOLATED` 节点不应主动拒绝 `notify` 请求——即使处于孤立状态，收到其他节点的 `notify` 也应正常处理，因为这可能是网络恢复后第一个“重连信号”。

第六章

边缘情况与处理策略

6.1 并发加入（Concurrent Join）

问题描述：多个节点几乎同时加入网络时，它们可能都通过同一个 `seed` 节点执行 `find_successor`，并被分配到相邻的区间，导致 `successor/predecessor` 关系在短时间内出现不一致（例如：A 认为 B 是其 `successor`，同时 C 也插入在 A 和 B 之间但尚未被 A 发现）。

处理方式：Chord 协议的设计天然支持并发加入，原因在于 `stabilize` 和 `notify` 是幂等的“最终一致”协议。在多个节点并发加入后的若干个维护周期内，`stabilize` 会逐步发现并纠正所有不一致的 `successor/predecessor` 关系。具体而言，每轮 `stabilize` 至少能修复一对邻居关系，经过 $O(\log N)$ 轮后网络达到稳定状态。**不需要分布式锁**，因为 `notify` 的幂等性保证了重复执行不会引入新的错误状态。

6.2 节点 ID 碰撞

问题描述：两个不同的节点 URI 在极低概率下产生相同的 SHA-1 哈希值（ 2^{160} 空间内的随机碰撞）。

处理方式：节点 N 在发起 join 请求时，seed 节点（或其路由到的目标 successor）会检查自身的 ID 是否与 N 相同。若发现碰撞，返回 409 ID_COLLISION 错误，N 记录错误日志并停止加入。实践中，SHA-1 在 160 位空间内的碰撞概率约为 $1 / 2^{160}$ ，在学习场景下完全可以忽略。

6.3 网络分区（Network Partition）

问题描述：网络发生分裂，原有的 Chord 环被分成两个或多个独立子集，每个子集各自形成独立的 Chord 环。

处理方式：分区期间，每个子环各自独立运行，内部路由正常。分区恢复后，两个子环的节点通过 stabilize 和 notify 逐渐“发现”对方的节点，经过若干维护周期后合并为一个统一的环。合并过程中存在短暂的路由不一致（某些键的查找可能返回错误节点），这在最终一致性模型下是可接受的。Tracker 可通过对比其全局节点列表和各节点上报的拓扑信息，辅助人工检测分区事件。

6.4 Tracker 不可用

影响范围与处理策略如下：

- **现有 ACTIVE 节点：**完全不受影响。report_to_tracker 静默失败，不影响任何 Chord 协议逻辑。
- **新加入节点：**无法获取 seed 节点列表，需通过手动配置 seed 节点地址（在启动参数或配置文件中直接指定任意已知节点的 `https://hostname`）绕过 Tracker。
- **监控面板：**失去全局统计视图，但可以直接查询各节点的 `/chord/state` 接口获取本地路由状态。

6.5 环中只有一个节点

单节点环是 Chord 网络的最简特殊情况，需确保协议在此情况下不产生死循环或错误：

- `predecessor = None`，`successor = self`，`successor_list = [self]`。
- 任意 `find_successor(id)` 均返回 `self`（因为 `self` 是唯一节点，负责全部键空间）。
- `stabilize`：向 `self` 请求 `predecessor`（本地直接返回 `None`），然后向 `self` 发送 `notify`（本地处理）。`notify` 逻辑：`predecessor == None`，但发起方也是 `self`，接受后 `predecessor = self`（或保持 `None`，避免 `self` 指向自己形成自引用，实现时可选择保持 `predecessor = None`）。
- 第一个新节点 X 加入时：X 向 `self` 发起 `join`，`self` 返回 `successor = self`，X 设置 `successor = self` 并发 `notify` 给 `self`。`self` 在下一次 `stabilize` 中发现 X 成为新的前驱，并将 X 纳入 `successor_list`。

6.6 手指表中存在过期节点

问题描述：手指表条目中记录的节点已经离线，但尚未被 `fix_fingers` 修复（`fix_fingers` 轮转修复，最长 160 分钟才会完整刷新一遍）。

处理方式：路由时遇到无响应的手指表节点，跳过该条目，尝试手指表中下一个候选（即 `i-1` 的条目）。同时将该节点标记为 `suspicious`，下一次 `fix_fingers` 轮转到该条目时会执行 `find_successor` 并修复。在大量手指表条目失效的极端情况下，路由退化为 $O(N)$ 的线性后继链查找，直到 `fix_fingers` 逐步修复。

6.7 find_successor 中的 hop 循环

问题描述：在环部分断裂时，closest_preceding_node 的选择可能使请求在少数几个节点之间循环（节点 A 转发给 B，B 转发给 A，...）。

处理方式：

- 迭代模式下，发起方维护 visited_nodes 集合，每跳将当前节点 ID 加入集合。若下一跳节点已在集合中，立即返回错误（环路检测）。
- max_hops = 161 作为硬上限，即使 visited_nodes 检测失效，也能终止请求。

6.8 节点加入时的 successor 选择

问题描述：新节点选择的 seed 节点恰好处于 LEAVING 状态，其 find_successor 响应可能不准确，导致新节点加入到错误的位置。

处理方式：Tracker 返回 k=5 个 seed 候选节点，新节点按顺序尝试每个候选，直到找到一个返回正常响应（状态码 200，且响应中包含有效 successor 信息）的 seed 节点。LEAVING 状态的节点在收到 join 请求时返回 503 NODE_LEAVING，新节点收到后自动尝试下一个 seed。

6.9 Stabilize 中的竞态

问题描述：节点 A 正在执行 stabilize 并准备更新其 successor，此时节点 B 也在执行 stabilize，两者对同一区间的邻居关系可能产生短暂冲突。

处理方式：Chord 协议的论文证明，在最终一致性模型下，stabilize + notify 的并发执行是安全的，不会导致永久性的错误状态。短暂的不一致（如 A 的 successor 指向 C 而 B 的 predecessor 指向 D）会在后续的维护周期中自动收敛。实现时，每个节点对自身状态的修改使用读写锁保证单节点内的并发安全，但跨节点的分布式状态不依赖分布式锁。

6.10 大量节点同时失效（批量崩溃）

问题描述：超过 $r=3$ 个连续节点同时崩溃，导致某个节点的整个 successor_list 全部失效，进而引发环断裂。

处理方式与分析：

- 这是 Chord 的已知局限性——后继列表只能容忍最多 r 个连续节点同时失效。
- 概率分析：在 N 个节点的网络中， $r=3$ 个特定连续节点同时崩溃的概率极低，且随着 r 增大呈指数级降低。
- 处理策略：受影响的节点切换为 ISOLATED 状态，通过 Tracker 获取新的 seed 节点，执行完整 join 流程重新加入。
- 扩展建议：若需要更强的容错性，可将 r 增大到 5 或 10（参见 10.1 节第 4 条实验方向）。

6.11 TLS 证书问题

问题描述：节点证书过期、域名不匹配、自签名证书未被 CA 信任，或中间证书缺失，导致 TLS 握手失败。但节点进程本身可能完全正常运行。

处理策略：

- TLS 错误不触发 `consecutive_failures` 增加，避免因证书配置问题被错误踢出网络
- 单独维护 `tls_error_count`，连续 TLS 错误达到阈值（默认 3 次）时写入告警日志，并通过 heartbeat 上报给 Tracker
- 节点启动时应自检证书有效期，若距过期不足 7 天则打印警告日志
- 建议使用 Let's Encrypt 等自动续签机制，或由内部统一 CA 签发证书并配置监控告警

6.12 高延迟网络（跨地域节点）

问题描述：跨地域节点（如亚洲与欧洲之间）RTT 可能超过 150ms。`find_successor` 的迭代过程若经过多个跨域节点，总延迟可能逼近超时上限，导致手指表修复频繁失败。

处理策略：

- `fix_fingers` 使用独立的宽松超时（30 秒，见 5.2.2 节），允许跨地域的 `find_successor` 迭代完整完成
- 手指表修复失败时，仅将对应条目标记为 `health_status = unknown`，不视为节点失效，等待下一轮 `fix_fingers` 重试
- 可通过 Tracker 的 `avg_request_latency_ms` 统计观察全局延迟分布，据此调整 `chord.http_total_timeout_seconds` 配置值

6.13 节点 URI 变更

问题描述：节点运营者希望更换域名（如从 `node-old.example.com` 变更为 `node-new.example.com`）。

处理规则：URI 是节点的不可变标识符——URI 变更等同于旧节点下线、新节点加入，因为 `node_id = SHA1(uri)` 会随之改变，新旧节点在环上的位置完全不同。正确的操作流程为：

1. 旧 URI 节点执行 Graceful Leave（通知 `successor` 和 `predecessor`，向 Tracker 注销）
2. 新 URI 节点执行完整 Join 流程重新加入网络

不支持原地 URI 更新。试图在不执行 Leave 的情况下直接切换 URI 会导致旧节点 ID 的残留路由条目长期存在于其他节点的手指表中，直至 `fix_fingers` 逐渐覆盖。

6.14 HTTPS 重定向与反向代理

问题描述：节点前方的反向代理（如 Nginx、Caddy）可能对某些路径返回 301/302 重定向，导致 Chord 协议请求被路由到意外地址，或在重定向链中丢失请求体。

处理策略：

- 节点 HTTP 客户端默认**不自动跟随重定向**，避免被重定向到非 Chord 端点
- 3xx 响应视为配置错误，记录告警日志，不计入 `consecutive_failures`，不触发节点失效流程

- 节点运营者应确保反向代理对 `/chord/*` 和 `/tracker/*` 路径直接透传，不做重定向
- 若节点部署时配置了 HTTP→HTTPS 重定向，客户端应始终直接使用 `https://` URI，不发起 HTTP 请求，从而完全绕过重定向

RESTful API 规范

7.1 通用约定

本章定义系统中所有 RESTful API 的设计规范，节点 API 和 Tracker API 均遵循以下通用约定：

约定项	规定
内容类型	所有请求和响应均使用 <code>Content-Type: application/json</code>
Base URL（节点）	节点 URI 即为其 Base URL（例如 <code>https://node1.example.com</code> ），API 路径直接拼接其后
Base URL（Tracker）	<code>https://tracker.example.com</code>
HTTP 方法语义	GET（查询状态）、POST（执行操作或发送通知）、DELETE（主动注销/离网）
ID 字段格式	所有 <code>node_id</code> 字段均为 40 位十六进制字符串（小写），长度严格为 40 字符
时间戳格式	所有时间字段均使用 ISO 8601 UTC 格式，例如 <code>2026-05-27T13:00:00Z</code>
跳数控制字段	<code>hop_count</code> （整数），在迭代 <code>find_successor</code> 中由调用方维护，每次转发递增
null 字段	可选字段值为空时使用 JSON <code>null</code> ，而非省略字段
字符编码	UTF-8

7.2 统一错误响应格式

所有 API 在发生错误时，均返回以下统一结构的 JSON 响应体：

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "人类可读的错误描述（中文或英文均可）",
    "detail": {}
  }
}
```

HTTP 状态码	error.code	含义	常见触发场景
----------	------------	----	--------

400	INVALID_REQUEST	请求参数格式错误	node_id 长度不为 40、端口超范围、缺少必填字段
404	NODE_NOT_FOUND	请求的节点不存在	Tracker 中查询不存在的 node_id
409	ID_COLLISION	节点 ID 已存在	加入时发现 node_id 与现有节点冲突
503	NODE_ISOLATED	节点处于孤立状态，无法路由	节点 ISOLATED 时收到 find_successor 请求
503	MAX_HOPS_EXCEEDED	超过最大跳数限制	find_successor 的 hop_count 达到 161
503	NODE_LEAVING	节点正在离网，拒绝服务	节点处于 LEAVING 状态时收到任何非 ping 请求
504	UPSTREAM_TIMEOUT	上游节点超时	递归模式下（如有）上游节点 5 秒内无响应

7.3 节点 API (Node API)

API 1: GET /chord/identity

GET /chord/identity — 查询节点身份信息

用途：返回本节点的基本身份信息，不含路由表详情。用于快速确认节点存在和状态。

请求：无参数，无请求体。

成功响应 (200)：

```
{
  "node_id":    "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "uri":       "https://node1.example.com",
  "status":    "ACTIVE",
  "joined_at": "2026-05-27T12:00:00Z"
}
```

特别说明：即使节点处于 ISOLATED 状态，也应正常响应此接口（返回 status: "ISOLATED"），用于诊断和被其他节点发现。

API 2: GET /chord/state

GET /chord/state — 查询完整 Chord 状态

用途：返回完整的 Chord 节点状态，包括路由表全量数据，用于调试和监控。

请求：无参数。

成功响应 (200)：

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "status": "ACTIVE",
  "predecessor": { "node_id": "...", "uri": "https://node4.example.com" },
  "successor": { "node_id": "...", "uri": "https://node2.example.com" },
  "successor_list": [
    { "node_id": "...", "uri": "https://node2.example.com" },
    { "node_id": "...", "uri": "https://node3.example.com" },
    { "node_id": "...", "uri": "https://node4.example.com" }
  ],
  "finger_table": [
    { "index": 0, "start": "...", "node": { "node_id": "...", "uri": "https://nodeX.example.com" }, "status": "ok" },
    { "index": 1, "start": "...", "node": { "node_id": "...", "uri": "https://nodeX.example.com" }, "status": "ok" }
  ],
  "last_maintenance_at": "2026-05-27T13:00:00Z",
  "next_finger_index": 42
}
```

说明： finger_table 返回全部 160 个条目 (index 0 ~ 159)。predecessor 为空时值为 JSON null (字段不省略)。

API 3: GET /chord/ping

GET /chord/ping — 节点存活检查

用途： 轻量级存活探测，用于 check_predecessor 和 health_check_ring。

请求： 无参数。

成功响应 (200)：

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "status": "ACTIVE",
  "timestamp": "2026-05-27T13:00:00Z"
}
```

说明： 响应必须在 5 秒内返回，否则调用方认为该节点失效。ISOLATED 节点也应响应 ping (返回 status: "ISOLATED")，表示节点存在但无法参与路由。

API 4: POST /chord/find_successor

POST /chord/find_successor — 迭代查找后继节点

用途：Chord 路由的核心接口，迭代式查找给定 ID 的后继节点。

请求体：

```
{
  "id":          "b94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "hop_count":    0,
  "max_hops":     161,
  "visited_nodes": ["a94a8fe5ccb19ba61c4c0873d391e987982fbbd3"]
}
```

字段	类型	说明
id	string	目标 ID，40 位十六进制字符串
hop_count	integer	当前跳数，发起方初始为 0
max_hops	integer	最大允许跳数，固定为 161 (m+1)
visited_nodes	array	已访问节点 ID 列表，用于环路检测

成功响应 (200) - 直接找到后继 (found: true)：

```
{
  "found":      true,
  "successor": { "node_id": "c84a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
"https://node2.example.com" },
  "hop_count":  3
}
```

成功响应 (200) - 需要继续转发 (found: false，迭代模式)：

```
{
  "found":      false,
  "next_hop": { "node_id": "b84a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
"https://node3.example.com" },
  "hop_count":  1
}
```

错误响应：503 MAX_HOPS_EXCEEDED（跳数超限）、503 NODE_ISOLATED（节点孤立）、400 INVALID_REQUEST（id 格式错误）、503 NODE_LEAVING（节点离网中）。

重要：迭代模式说明

当返回 found: false 时，本节点不会主动调用 next_hop ——而是将 next_hop 返回给调用方，由调用方自行向 next_hop 节点发起下一次请求。调用方维护跳转循环，确保 HTTP 调用深度始终为 1。

API 5: GET /chord/predecessor

GET /chord/predecessor — 查询前驱节点

用途：返回本节点的前驱节点信息，由 stabilize 协议调用。

请求：无参数。

成功响应（200） - 前驱存在：

```
{
  "predecessor": { "node_id": "a04a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
    "https://node4.example.com" }
}
```

成功响应（200） - 前驱为空：

```
{
  "predecessor": null
}
```

说明：predecessor 为 None 时，返回 {"predecessor": null}，HTTP 状态仍为 200，不返回 404。

API 6: POST /chord/notify

POST /chord/notify — 通知前驱候选

用途：节点 N 通知本节点"我可能是你的前驱"，由 stabilize 协议触发。

请求体：

```
{
  "node": { "node_id": "a04a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
    "https://node4.example.com" }
}
```

成功响应（200） - 接受：

```
{
  "accepted": true,
  "predecessor": { "node_id": "a04a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
    "https://node4.example.com" }
}
```

成功响应（200） - 拒绝：

```
{
  "accepted": false,
  "predecessor": { "node_id": "当前前驱的 node_id", "uri": "https://nodeX.example.com" }
}
```

说明： `accepted: true` 表示请求方已被接受为新前驱；`accepted: false` 表示请求方不在合法前驱范围内，`predecessor` 不变。节点处于 `LEAVING` 状态时返回 `503 NODE_LEAVING`。

API 7: GET /chord/successor_list

GET /chord/successor_list — 查询后继列表

用途： 返回本节点的后继列表，由 `stabilize` 协议在获取 `successor` 的 `successor_list` 时调用。

请求： 无参数。

成功响应 (200)：

```
{
  "successor_list": [
    { "node_id": "...", "uri": "https://node2.example.com" },
    { "node_id": "...", "uri": "https://node3.example.com" },
    { "node_id": "...", "uri": "https://node4.example.com" }
  ]
}
```

说明： 列表长度 1 到 $r=3$ ，按环上距离从近到远排序（第 0 项最近）。单节点环时返回 `[self]`，列表长度为 1。

API 8: POST /chord/join

POST /chord/join — 通过本节点加入网络

用途： 新节点 N 通过本节点（作为 `seed/bootstrap` 点）加入 Chord 网络，获取初始 `successor` 信息。

请求体：

```
{
  "node": { "node_id": "f04a8fe5ccb19ba61c4c0873d391e987982fbbd3", "uri":
    "https://node5.example.com" }
}
```

成功响应 (200)：


```
{
  "successor": { "node_id": "...", "uri": "https://node2.example.com" },
  "successor_list": [
    { "node_id": "...", "uri": "https://node2.example.com" }
  ]
}
```

说明：本接口内部调用 `find_successor(N.id)`，返回 N 的初始 `successor` 及其 `successor_list`。N 收到后自行完成状态设置并向 `successor` 发送 `notify`。若本节点处于 `ISOLATED` 或 `LEAVING` 状态，返回 503。**碰撞检测：**若 `N.node_id == self.node_id`，返回 409 `ID_COLLISION`。

API 9: POST /chord/leave

POST /chord/leave — 接收邻居离网通知

用途：Graceful Leave 协议中，离网节点通知其 `successor` 或 `predecessor`，告知邻居关系变更。

请求体（离网节点通知其 `successor`）：

```
{
  "role": "predecessor_leaving",
  "new_predecessor": { "node_id": "...", "uri": "https://nodeX.example.com" }
}
```

请求体（离网节点通知其 `predecessor`）：

```
{
  "role": "successor_leaving",
  "new_successor": { "node_id": "...", "uri": "https://nodeX.example.com" }
}
```

成功响应（200）：

```
{ "acknowledged": true }
```

说明：收到此通知后，节点立即更新对应邻居关系，并触发一次提前的 `stabilize`（不必等待下一个 60 秒周期），以加速拓扑修复。

API 10: GET /chord/finger_table

GET /chord/finger_table — 查询手指表（调试）

用途：返回完整的手指表，用于调试和路由正确性验证。

请求：无参数。

成功响应（200）：

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "finger_table": [
    {
      "index": 0,
      "start": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd4",
      "node": { "node_id": "...", "uri": "https://nodeX.example.com" },
      "status": "ok"
    }
  ],
  "next_repair_index": 42
}
```

finger 条目 status 字段说明：

status 值	含义
ok	条目已成功验证，节点正常响应
suspicious	最近一次请求失败，可能临时不可达
unknown	条目尚未经过 fix_fingers 验证（初始状态）

7.4 Tracker API

Tracker 服务提供以下 8 个 API 接口，供节点注册、心跳上报、bootstrap 引导及监控使用。

Tracker API 1：POST /tracker/nodes

POST /tracker/nodes — 节点注册

用途：节点向 Tracker 注册自身，通常在完成 join 流程后调用。

请求体：

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "uri": "https://node1.example.com"
}
```

成功响应（200）：

```
{
  "registered":      true,
  "known_nodes_count": 15,
  "message":         "Node registered successfully"
}
```

说明：若该 node_id 已存在，则更新 last_seen 和连接信息（允许节点重启后重新注册）。Tracker 存储上限可配置（默认 1000），超过时淘汰最久未活跃的节点。

Tracker API 2：DELETE /tracker/nodes/{node_id}

DELETE /tracker/nodes/{node_id} — 节点注销

用途：节点主动向 Tracker 注销，在 Graceful Leave 流程中调用。

请求：无请求体，node_id 在 URL 路径中。

成功响应（200）：

```
{
  "deregistered": true,
  "node_id":      "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3"
}
```

错误响应：404 NODE_NOT_FOUND（该 node_id 未在 Tracker 注册）。

Tracker API 3：GET /tracker/nodes/seeds

GET /tracker/nodes/seeds — 获取 bootstrap seed 节点

用途：新节点获取 bootstrap 所需的 seed 节点列表。

请求参数（Query String）：

参数	类型	默认值	说明
count	integer	5	请求的 seed 节点数量，最大 20
exclude	string	—	逗号分隔的 node_id 列表，排除这些节点（避免返回自己或已知失效节点）

示例请求： GET /tracker/nodes/seeds?count=5&exclude=a94a8fe5ccb19ba61c4c0873d391e987982fbbd3

成功响应（200）：

```
{
  "seeds": [
    { "node_id": "...", "uri": "https://nodeX.example.com" },
    { "node_id": "...", "uri": "https://nodeX.example.com" }
  ],
  "total_known": 15,
  "note": "Nodes selected randomly from active list"
}
```

说明：Tracker 随机选取，倾向于返回 last_seen 时间较新的节点。若 total_known 为 0，返回空 seeds 列表，表示当前无已知节点，该节点将自成单节点环。

Tracker API 4: POST /tracker/nodes/{node_id}/heartbeat

POST /tracker/nodes/{node_id}/heartbeat — 心跳上报

用途：节点定期向 Tracker 汇报存活状态和本地统计信息，每维护周期调用一次（约每分钟）。

请求体：

```
{
  "status": "ACTIVE",
  "successor_id": "b94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "predecessor_id": "994a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "successor_list_size": 3,
  "finger_table_coverage": 0.85,
  "uptime_seconds": 3600,
  "maintenance_cycles": 60
}
```

字段	说明
finger_table_coverage	手指表中已成功更新条目的比例（0.0 到 1.0）
maintenance_cycles	自节点启动以来执行的维护周期总次数
successor_id	当前直接后继的 node_id（可为 null）
predecessor_id	当前前驱的 node_id（可为 null）

成功响应（200）：

```
{
  "acknowledged": true,
```

```
"tracker_time": "2026-05-27T13:00:00Z"
}
```

Tracker API 5: GET /tracker/nodes

GET /tracker/nodes — 查询全部节点列表

用途：获取 Tracker 已知的全部节点列表，供管理和调试使用。

请求参数 (Query String)： `status` (过滤状态，如 ACTIVE)、`limit` (默认 50，最大 200)、`offset` (默认 0，分页)。

成功响应 (200)：

```
{
  "nodes": [
    {
      "node_id": "...",
      "uri": "https://nodeX.example.com",
      "status": "ACTIVE",
      "joined_at": "2026-05-27T12:00:00Z",
      "last_seen": "2026-05-27T13:00:00Z",
      "report_count": 42
    }
  ],
  "total": 15,
  "limit": 50,
  "offset": 0
}
```

Tracker API 6: GET /tracker/nodes/{node_id}

GET /tracker/nodes/{node_id} — 查询特定节点记录

用途：查询特定节点在 Tracker 中的完整记录。

请求：无请求体，`node_id` 在 URL 路径中。

成功响应 (200)：返回单个 TrackerNodeRecord 对象 (结构同 API 5 的 nodes 数组元素)。

错误响应：404 NODE_NOT_FOUND。

Tracker API 7: GET /tracker/stats

GET /tracker/stats — 全局健康统计

用途：获取 DHT 网络全局健康统计，供监控面板使用。

请求：无参数。

成功响应（200）：

```
{
  "total_nodes":          15,
  "active_nodes":         13,
  "isolated_nodes":       1,
  "leaving_nodes":        1,
  "stale_nodes":          2,
  "avg_finger_table_coverage": 0.82,
  "avg_uptime_seconds":    1800,
  "oldest_node_joined_at": "2026-05-27T10:00:00Z",
  "newest_node_joined_at": "2026-05-27T13:00:00Z",
  "tracker_uptime_seconds": 7200,
  "stats_generated_at":    "2026-05-27T13:00:00Z"
}
```

字段	说明
stale_nodes	超过 180 秒（3 分钟）未发送 heartbeat 的节点，可能已崩溃但未注销
avg_finger_table_coverage	所有上报节点的手指表覆盖率平均值

说明：Tracker 不主动探测节点存活，统计数据完全依赖节点的 heartbeat 上报，因此 stale_nodes 计数存在滞后性（最多滞后 180 秒）。

Tracker API 8：GET /tracker/health

GET /tracker/health — Tracker 自身健康检查

用途：检测 Tracker 服务自身是否正常运行，用于运维监控和新节点 bootstrap 前的可用性检测。

请求：无参数。

成功响应（200）：

```
{
  "status":          "ok",
  "uptime_seconds":  7200,
  "timestamp":       "2026-05-27T13:00:00Z"
}
```

消息时序图（文字描述）

8.1 新节点加入网络时序

以下描述新节点 N 通过 Tracker 和 seed 节点 S1 加入已有网络的完整消息时序（A 为 N 的初始 successor）：

步骤	发起方 → 接收方	操作	说明
1	N 本地	计算 node_id = SHA1(uri)（对节点 URI 规范化后执行 SHA-1），初始化数据结构，状态 = INITIALIZING	本地操作，无网络交互
2	N → Tracker	GET /tracker/nodes/seeds?count=5	请求 seed 节点列表
3	Tracker → N	200 OK，返回 [S1, S2, S3, S4, S5]	Tracker 返回随机选取的活跃节点
4	N 本地	状态切换为 JOINING	开始 join 流程
5	N → S1	POST /chord/join（传入 N 的 NodeInfo）	通过第一个 seed 节点引导
6	S1 内部	执行 find_successor(N.id)，可能涉及多个节点间的迭代查找	S1 作为 bootstrap 点，代替 N 完成路由查找
7	S1 → N	200 OK，返回 successor=A，successor_list=[A, B, C]	N 获得初始 successor 和后继列表
8	N 本地	设置 successor=A，successor_list=[A,B,C]，predecessor=None	本地状态更新
9	N → A	POST /chord/notify（传入 N 的 NodeInfo）	通知 A：N 可能是你的前驱
10	A 内部	检查：N.id ∈ (A.predecessor.id, A.id)?；若是：A.predecessor = N	A 更新自己的前驱关系
11	A → N	200 OK，{accepted: true, predecessor: N}	A 确认接受 N 为新前驱
12	N 本地	状态切换为 ACTIVE，启动周期维护调度器	N 正式加入网络
13	N → Tracker	POST /tracker/nodes（注册）+ POST /tracker/nodes/{N.id}/heartbeat（首次心跳）	向 Tracker 登记
14	A 在下一维护周期	执行 stabilize，发现 A.predecessor = N，向 N 发送 notify；N 通过 notify 确认 A 为其 successor	拓扑关系在维护周期中进一步稳定

8.2 节点主动离网时序

节点 N 主动下线，其 successor 为 A，predecessor 为 P：

步骤	发起方 → 接收方	操作	说明
1	N 本地	收到关闭信号，状态切换为 LEAVING	停止接受新的路由请求
2	N → A (successor)	POST /chord/leave, {role: "predecessor_leaving", new_predecessor: P}	告知 A：其新前驱应为 P
3	A 内部	更新 predecessor = P，立即触发提前 stabilize	A 的拓扑关系快速修复
4	A → N	200 OK, {acknowledged: true}	
5	N → P (predecessor)	POST /chord/leave, {role: "successor_leaving", new_successor: A}	告知 P：其新后继应为 A
6	P 内部	更新 successor = A，立即触发提前 stabilize	P 的拓扑关系快速修复
7	P → N	200 OK, {acknowledged: true}	
8	N → Tracker	DELETE /tracker/nodes/{N.id}	从 Tracker 注销
9	N 本地	停止 HTTP 服务，进程退出	离网完成

8.3 节点崩溃后的恢复时序

节点 C 意外崩溃（无 Graceful Leave），其 predecessor 为 P，successor 为 S：

时间线	事件	说明
T=0	节点 C 崩溃（进程终止）	无通知发出，邻居节点不知情
T=60s（第 1 次维护周期）	S 执行 check_predecessor：Ping C → 超时（5s）→ S.predecessor = None	S 清空对 C 的前驱引用
T=60s（第 1 次维护周期）	P 执行 stabilize：询问 C（其 successor）的 predecessor → 超时 → 标记 C 为 suspicious	P 发现直接后继无响应，但尚未切换
T=120s（第 2 次维护周期）	P 执行 stabilize：C 仍无响应（第 2 次失败）→ 标记 C 为 failed → 切换 successor = successor_list[1]（即 S 或 S'）	P 正式放弃 C，使用备份后继
T=120s	P 向新 successor 发送 notify(P)，新 successor 检查：P.id ∈ (new_successor.predecessor.id, new_successor.id)?	重新建立 P 与新 successor 之间的前驱关系
T=120s	新 successor.predecessor = P，拓扑修复完成	约 2 个维护周期（约 2 分钟）完成自动修复
T+后续	Tracker 在约 T+180s 后将 C 标记为 stale（heartbeat 停止超过 180s）	Tracker 从活跃节点列表中淡出 C

配置参数参考

以下是系统所有可配置参数的完整清单，建议通过配置文件（YAML 或 JSON）管理，支持环境变量覆盖。所有参数均提供合理的默认值，学习场景下可直接使用默认配置。

参数名	默认值	类型	说明
chord.m	160	integer	ID 空间位数，决定手指表长度和 ID 范围。与 SHA-1 输出长度一致，不建议修改。
chord.successor_list_size	3	integer	后继列表长度 r。增大 r 可提升容错性，但增加每次 stabilize 的通信量。
chord.maintenance_interval_seconds	60	integer	维护周期间隔（秒）。减小此值可加快拓扑收敛，但增加网络负载；增大此值反之。
chord.http_timeout_seconds	5	integer	单次 HTTP 请求超时（秒）。超过此时间视为失败，不重试。
chord.max_hops	161	integer	find_successor 最大跳数（m+1 = 161）。超过此值立即返回 MAX_HOPS_EXCEEDED 错误。
chord.suspicious_threshold	1	integer	将节点标记为 suspicious 的连续失败次数。达到此次数后降低路由优先级。
chord.failed_threshold	3	integer	将节点标记为 failed 的连续失败次数。达到此次数后从路由表中移除该节点。
tracker.url	null	string	Tracker 服务地址（如 http://tracker.local:9000）。null 表示不使用 Tracker，需手动配置 seed 节点。
tracker.seed_count	5	integer	从 Tracker 请求的 seed 节点数量。节点依次尝试每个 seed，直到成功或列表耗尽。
tracker.heartbeat_interval_seconds	60	integer	心跳上报间隔（秒），建议与 maintenance_interval_seconds 保持一致。

tracker.stale_threshold_seconds	180	integer	Tracker 判定节点为 stale 的心跳超时阈值（秒）。超过此时间未上报 heartbeat 的节点被标记为 stale。
tracker.max_nodes	1000	integer	Tracker 存储的最大节点数。超过时淘汰 last_seen 最旧的节点记录。
node.uri	https://node1.example.com	string	节点自身的 HTTPS URI，用于计算 node_id 并对外通告。须规范化：scheme 和 host 小写，去除尾部斜杠，非 443 端口须保留。
node.manual_seeds	[]	array	手动配置的 seed 节点 URI 列表（格式：["https://node1.example.com", "https://node2.example.com"]），在无 Tracker 时使用。

学习扩展建议

10.1 下一步实验方向

在完成基础 Chord 协议实现并验证其正确性后，以下六个方向可作为进阶学习实验，每个实验均能加深对分布式系统某一方面的理解：

- 可视化环形拓扑：**实现一个简单的 Web UI，通过定期调用 Tracker 的 /tracker/nodes 接口获取所有节点状态，在浏览器中绘制 Chord 环（圆形分布图，标注每个节点的 node_id 前 8 位和位置角度）。可以实时展示节点的加入、离开和 ISOLATED 状态变化，直观理解环形路由结构。建议使用 HTML Canvas 或 SVG 实现，数据通过轮询（每 5 秒刷新一次）获取。
- 手指表验证器：**编写一个独立的测试工具，遍历网络中所有节点（通过 Tracker 获取列表），对每个节点的手指表进行正确性验证——即验证每个 finger[i].node 是否真的是 finger[i].start 的正确 Chord 后继。可以通过对比"理论上正确的后继"与"实际记录的后继"来定量评估手指表的准确率，并定位路由错误的根源。
- 模拟节点崩溃与恢复观测：**在运行一个多节点的 Chord 网络后，强制 kill 某个节点的进程（模拟崩溃），然后观察并记录网络修复过程：邻居节点何时检测到崩溃（第几个维护周期）、后继切换何时完成、前驱关系何时重新建立。将观测结果与 8.3 节的时序描述进行对比，验证协议行为的正确性。
- 扩大后继列表（调整 r 值）：**将 chord.successor_list_size 从默认的 r=3 分别调整为 r=5 和 r=10，然后模拟同时 kill 4 个连续节点的场景，观察不同 r 值下网络的表现：r=3 时网络断裂，r=5 时仍可自愈。通过实验直观理解 r 与容错性之间的权衡关系，以及增大 r 对 stabilize 通信量的影响。

5. **加入 KV 存储，实现完整 DHT**：在 ACTIVE 节点基础上实现数据存储接口：PUT /data/{key}（将 key 路由到正确的 responsible 节点并存储 value）和 GET /data/{key}（路由查找 key 的 responsible 节点并返回 value）。这需要在 find_successor 的基础上加入数据转发逻辑，完成从"纯组网 DHT"到"完整分布式哈希表"的进化。可进一步实验副本存储（将同一键值存储在后继列表中的多个节点上）。
6. **消息追踪与路由效率分析**：在 find_successor 请求中加入 trace_id 字段，每一跳记录经过的节点 ID 和时间戳，最终将完整路由路径返回给发起方。收集多次查找的跳数统计，验证实际查找效率是否符合 $O(\log N)$ 的理论预期——例如，在 $N=8$ 个节点的网络中，平均跳数应约为 $3 (\log_2 8)$ 。通过增减节点数量，观察跳数随 N 变化的趋势曲线。

10.2 与真实 DHT 系统的主要差异

本学习系统与生产级 DHT 实现（如 BitTorrent DHT / Kademlia、IPFS 的 libp2p-kad-dht）存在若干重要差异。理解这些差异有助于学习者将所学知识迁移到真实系统的理解和开发中：

差异维度	本学习系统	真实生产系统	影响
传输层协议	HTTP/1.1 (TCP, 文本协议)	UDP (Kademlia) 或 QUIC (libp2p)	HTTP 延迟较高 (TCP 握手)，但实现简单、调试工具丰富。UDP 延迟低，但需自行处理丢包、重传和消息完整性验证。
路由算法	Chord (顺时针后继, 手指表)	Kademlia (XOR 距离度量, k-bucket)	Kademlia 的双向路由 (对称距离) 和 k-bucket 结构在实践中更高效；Chord 的线性环结构更易于理解和教学。
节点 ID 生成	SHA1(uri), 由节点 URI 规范化后计算	使用公钥哈希或 CA 颁发 (防 Sybil 攻击)	本系统节点 ID 由其 URI 唯一决定 (URI 固定则 ID 固定)，生产系统通过密码学手段防止节点 ID 被恶意选择 (Sybil 攻击)。
NAT 穿透	不处理, 假设节点直接可达	STUN/TURN、hole punching、中继	真实互联网上绝大多数节点位于 NAT 后，无法直接接受入站连接，需要专门的 NAT 穿透机制。
数据复制	不涉及 (无数据存储)	数据存储在多个最近节点 (k 副本)	生产 DHT 将数据存储 in 负责节点及其 k 个邻居上，节点离网时数据不丢失；需要定期 refresh 机制防止数据因节点流动而丢失。
安全性	无认证、无加密	TLS/noise 协议加密、节点身份认证	真实系统需防止中间人攻击、数据篡改和路由污染攻击 (恶意节点返回错误路由信息)。
Tracker/Bootstrap	中心化 Tracker (可选)	硬编码 bootstrap 节点 + DHT 自举	BitTorrent DHT 使用硬编码的少量 bootstrap 节点 (全球公开)，IPFS 使用类似机制，不依赖单一中心化 Tracker。

总体而言，本学习系统的设计刻意选择了"简洁优先"的方向：用 HTTP 替代 UDP、用 Chord 替代 Kademlia、用中心化 Tracker 替代完全去中心化 bootstrap，使协议语义更容易被独立观察和验证。在理解本系统的协议后，学习者可以尝试阅读 Kademlia 原论文 (Maymounkov & Mazières, 2002) 以及 libp2p 的实现代码，理解生产级系统如何在此基础上应对真实网络的复杂性。

文档版本信息

文档版本	v1.0（正式发布）
设计日期	2026 年 5 月 27 日
适用范围	学习用 Chord DHT 实现，面向分布式系统学习工程师
生产适用性	本文档描述的系统设计不适合直接用于生产环境，缺少安全性、性能优化和 NAT 穿透等生产级要求
核心参数	$m = 160$ （SHA-1）， $r = 3$ （后继列表），维护周期 = 60 秒，最大跳数 = 161
参考文献	Stoica et al., "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications", ACM SIGCOMM 2001